

Модуль 1. “Мови в програмуванні”

дисципліни

“Розробка компіляторів для предметно-орієнтованих мов”

Лектор: *проф. Григорій ЖОЛТКЕВИЧ, д.т.н.*

Зміст

1	Навіщо людині мова?	1
2	Роль мов у програмуванні	2
3	Мови загального призначення	3
4	Один приклад	4
5	Предметно-орієнтовані мови	7
5.1	Які мови називають предметно-орієнтованими?	7
5.2	Чому використовують предметно-орієнтовані мови?	7
5.3	Особливості предметно-орієнтованих мов та принципи їх дизайну	8
5.4	Два типи предметно-орієнтованих мов	9
6	Висновки	9
	Література	12

Тема “Мови у програмуванні” є вступом до предмету “Розробка компіляторів для предметно-орієнтованих мов”.

Викладення починається зі з’ясування питання “Навіщо людині мова?”. Підкреслюється, що мова відіграє невід’ємну роль у когнітивних процесах: структуруванні думок, абстрагуванні, зберіганні інформації та розвитку свідомості. Мова є унікальним інструментом комунікації, що формує основу людської цивілізації, забезпечуючи абстрактне мислення, гнучкість, передачу культурних надбань та об’єднання людей. Без мови було б неможливо утворити складні соціальні структури, розвинути науку чи технології.

Далі обговорюється роль мов у програмуванні, де вони постають як формальний засіб взаємодії людини з комп’ютером – своєрідний міст між людським інтелектом та обчислювальною потужністю машин. У цьому контексті обговорюється ряд ключових функцій, які мови програмування виконують.

Виходячи з цих функцій виділяється два основні класи мов програмування:

- Мови загального призначення (General-Purpose Languages, GPL). Вони характеризуються повнотою за Тьюрінгом, що дозволяє їм виражати будь-який алгоритм. Вони є універсальними та гнучкими, призначеними для широкого спектра завдань у різних предметних областях. Прикладами таких мов є Python, Java, C++, JavaScript, C#, Go та Rust. Незважаючи на сильні сторони, мови загального призначення мають і слабкості, такі як обмежені можливості універсального автоматичного аналізу коду, відсутність вбудованих механізмів для представлення доменної логіки та значний семантичний розрив між кодом і розумінням експертів предметної області.
- Предметно-орієнтовані мови (Domain-Specific Languages, DSL). Ці мови виникають як відповідь на слабкості мов програмування загального призначення, пропонуючи спеціалізацію для конкретної прикладної області. Предметно-орієнтовані мови використовують термінологію та концепції, знайомі фахівцям домену, що робить їх більш виразними, підвищує продуктивність розробників та спрощує обслуговування коду. Багато предметно-орієнтованих мов не є повними за Тьюрінгом (наприклад, SQL), що є свідомим обмеженням для спрощення та оптимізації. Їх дизайн орієнтований на потреби доменних експертів.

Розрізняють два типи предметно-орієнтованих мов: внутрішні (Embedded DSL, eDSL), реалізовані як бібліотеки або фреймворки всередині "хост"мови загального призначення (наприклад, SQLAlchemy для Python) та зовнішні, що мають повністю новий, незалежний синтаксис та вимагають окремого компілятора або інтерпретатора (наприклад, SQL, CSS, TeX, Regex, Gherkin).

Зрештою, вибір на користь використання мови загального призначення або створення предметно-орієнтованої мови є надзвичайно складним рішенням, яке має ключове значення для успішності проекту. Сучасний фахівець у галузі комп’ютерних наук має бути готовим до створення мови, орієнтованої на певну предметну область, та всієї необхідної технологічної інфраструктури для її використання.

1 Навіщо людині мова?

Для людини мова є невід’ємною частиною когнітивних процесів. Ми формуємо думки, аналізуємо інформацію, плануємо дії, зберігаємо спогади – і все це відбувається за допомогою мовних структур. Мова дозволяє нам

- **структурувати думки**, надаючи рамки для організації ідей і понять;

- **абстрагуватися**, тобто працювати з концепціями, які не мають безпосереднього фізичного втілення;
- **зберігати та відтворювати інформацію**, допомагаючи кодувати знання для подальшого використання;
- **розвивати свідомість**, забезпечуючи внутрішній діалог, що є основою самоусвідомлення.

Без мови мислення було б значно обмеженішим, імовірно, зводилося б до примітивних образів і відчуттів.

Крім того, мова є унікальним інструментом спілкування людей. Вона не просто дозволяє нам обмінюватися інформацією, а й формує саму основу людської цивілізації. Уявіть собі світ без мови: ми не змогли б передавати складні ідеї, навчати нові покоління, будувати спільні проекти чи навіть просто висловлювати свої почуття.

Мова є унікальним засобом комунікації з наступних причин.

1. Слова є не просто звуками, а символами, що репрезентують об'єкти, ідеї, емоції. Ця здатність до символізації дозволяє нам говорити про те, чого немає тут і зараз, про минуле, майбутнє, про абстрактні концепції. І все це завдяки символічній природі мови.
2. Мова не обмежує нас у створенні нових слів, фраз, виразів, щоб точніше описувати світ навколо нас. Постійно розвиваючись мова адаптується до нових реалій, технологій, культурних змін. Отже, мова є гнучкою та адаптивною.
3. Мова є головним носієм культури. Казки, легенди, пісні, наукові трактати, філософські ідеї – все це передається з покоління в покоління завдяки мові. Мова зберігає колективну пам'ять суспільства, тобто забезпечує створення та передачу культурних надбань.
4. Спільна мова об'єднує людей, створює відчуття належності до певної спільноти, нації. Вона є фундаментом для формування соціальних груп, інститутів і навіть цілих держав.
5. Завдяки мові ми можемо не лише передавати знання, а й навчати інших складних навичок, пояснювати принципи, координувати дії у великих масштабах. А саме це відрізняє людське суспільство від інших видів спільнот.

Без мови ми були б не здатні утворити такі складні соціальні структури, як держави чи міста, не змогли б розвинути науку, мистецтво чи технології. Саме мова робить нас людьми, які можуть мислити, творити та спілкуватися на безпрецедентному рівні.

2 Роль мов у програмуванні

У світі програмування мова відіграє абсолютно іншу, але не менш важливу роль, ніж у людському спілкуванні. Тут це не просто засіб вираження думок чи комунікації між людьми, а **формальний засіб** взаємодії людини з комп'ютером. Фактично, мова програмування є мостом між людським інтелектом та обчислювальною потужністю машини.

Мова у контексті програмування виконує наступні ключові функції.

- Найфундаментальнішою функцією є **управління комп'ютером**. Мова програмування дозволяє розробникам писати чіткі та однозначні інструкції, які комп'ютер здатен виконати. Кожна команда, кожен оператор у мові програмування відповідає певній дії машина. Без програми – послідовності таких інструкцій – комп'ютер був би просто набором електронних компонентів.
- Наступною функцією є забезпечення механізму **абстрагування** для вирішення складних задач. Мови програмування надають такі механізми, що дозволяють розробникам працювати з високорівневими концепціями, не заглиблюючись у дрібні деталі апаратної архітектури. Замість того, щоб керувати окремими апаратними компонентами, програміст може використовувати такі поняття, як “змінна”, “функція”, “об’єкт” тощо. Це значно спрощує розробку та дозволяє зосередитися на логіці програми, а не на низькорівневих операціях.
- Важливою функцією є також **структурування даних**, тобто забезпечення засобів для організації та зберігання даних у пам’яті комп'ютера. Необхідність такої функції зумовлена тим, що програми розробляються саме заради обробки даних. Зазначені засоби можуть надавати можливості для представлення як даних простих типів, як-от числа чи текст, так і складніших структур, як-от масиви, списки, об’єкти тощо. Правильний вибір та визначення структур даних є критично важливим для ефективності та логічності програми.
- Наступною функцією є **формальна специфікація та реалізації алгоритмів** – покрокових інструкцій для вирішення певних задач. Кожна мова програмування надає синтаксичні конструкції (наприклад, цикли, умовні оператори), які дозволяють програмісту точно описати послідовність дій для досягнення бажаного результату.
- Несподіваною функцією є **спрощення спілкування між програмістами** – стандартний синтаксис та семантика мови дозволяють різним програмістам розуміти код один одного, співпрацювати над проектами, підтримувати та модифікувати вже існуючі програми. Добре написаний певною мовою та коментований код є сам по собі формою документації.
- Насамкінець відзначимо функцію **управління ресурсами комп'ютера**. Ця функція є комплементарною до функції управління комп'ютером з метою забезпечення виконання певної задачі і стосується забезпечує доступу до різноманітних ресурсів – пам’яті, процесору, файлової системи, мережових з’єднань, різноманітних зовнішніх пристроїв тощо. Без такого доступу неможливо уявити сучасний обчислювальний процес.

Отже, мова у програмуванні – це не просто набір слів, а систематизований, логічний і формальний спосіб створення, опису та виконання цифрових рішень. Без такого засобу неможливо було б уявити сучасний технологічний світ.

Світ мов програмування дуже великий – кількість таких мов за різними оцінками варіюється на сьогодні в діапазоні від 8 000 до 9 000, який постійно розширюється.

Головною спільною рисою мов, що використовуються в програмуванні, є те, що всі вони є **формальними мовами**. Цю концепцією ми з вами обговоримо в наших наступних лекціях.

3 Мови загального призначення

Одним з головних питань, яке постає під час аналізу тієї чи іншої мови програмування, є питання про те, чи задовольняє мова умові повноти за Тьюрінгом – будь-який алгоритм

може бути специфікований засобами такої мови, чи ні.

Мови програмування, що задовольняють умові повноти за Тьюрінгом, називають **мовами загального призначення** (*General-Purposed Languages, GPL*).

Мови програмування цього класу, розробляються для вирішення широкого спектра задач створення програмного забезпечення в різних **предметних областях (доменах)**. Вони є універсальними та гнучкими.

Звичайно, мови цього класу мають як сильні, так і слабкі сторони, які ми згрупуємо відповідно до таких аспектів – теоретичний, комунікаційний, технологічний та аспекту “порогу входження” та ефективності використання мови. Виходячи з такого групування сильні та слабкі сторони мов загального призначення подані у Табл. 1.

Наведемо приклади найбільш популярних мов загального призначення.

- Python – дуже популярна мова завдяки своїй простоті, читабельності та широкому застосуванню (веб-розробка, штучний інтелект/машинне навчання, наука про дані, автоматизація, скриптинг);
- Java – мова широко використовується для корпоративних систем, Android-додатків, веб-серверів;
- C++ – мова використовується для системного програмування, розробки ігор, високо-продуктивних додатків;
- JavaScript є основною мовою для веб-розробки на стороні клієнта, також використовується на стороні сервера (Node.js) та для мобільних додатків (React Native);
- C# – основна мова для розробки додатків на платформі .NET (Windows-додатки, веб, ігри з Unity);
- Go (Golang) мова розроблена Google для ефективної роботи з мережевими сервісами та розподіленими системами;
- Rust мова з’явилася як потужна альтернатива C++ для системного програмування, що сфокусована на безпечному використанні пам’яті.

І, хоча вивчення програмування базується саме на мовах загального призначення, слабкі сторони зазначені в Табл. 1 мають враховуватися для підвищення якості та ефективності розробки програмних систем.

4 Один приклад

Давайте спробуємо накреслити шлях подолання зазначених вище слабкостей на конкретно-му, широко відомому прикладі: обробці персистентних даних.

Програмування почалося як діяльність спрямована на управління комп’ютером заради виконання складних, як правило наукових, обчисленнями. В такій ситуації час життя даних обмежувався часом виконання програми – дані були тимчасовими (*transient data*). Програми, написані мовами загального призначення (наприклад, ранніми версіями Fortran чи C), ефективно працювали з такими даними, оскільки вони існували лише в оперативній пам’яті під час роботи програми.

З часом прийшло усвідомлення того, що деякі дані мають власну цінність і тому їх треба зберігати постійно (*persistent data*) – наприклад, інформацію про клієнтів, фінансові транзакції, виробничі запаси. Виникла потреба у надійному та ефективному зберіганні, пошуку та модифікації великих обсягів даних, які мали б пережити завершення роботи

Табл. 1: Сильні та слабкі сторони мов загального призначення

Сильні сторони	Слабкі сторони
Теоретичний аспект	
► Повнота за Тьюрінгом надає мові необмежену виразність та універсальність для вирішення різноманітних обчислювальних проблем. ► Мова імплементує всі базові принципи інформатики та обчислень (наприклад, алгоритми, структури даних, керування потоком виконання), що робить її чудовим інструментом для програмування як такого.	► Повноту мови тягне всі наслідки теореми Райса, що з практичної точки зору означає принципову неможливість створення універсальних алгоритмів для гарантованої перевірки будь-якої нетривіальної семантичної властивості програм (наприклад, чи зупиниться вона, чи виникне ділення на нуль в процесі виконання). Це обмежує можливості автоматичного аналізу, верифікації та оптимізації коду. ► Мова не забезпечує вбудовані механізми для коректного представлення доменної логіки на синтаксичному рівні, тобто усі доменні правила мають бути закодовані та перевірені вручну, що збільшує ймовірність логічних помилок.
Комунікаційний аспект	
► Мова надає розробнику гнучкість для створення будь-яких моделей даних та поведінки, необхідних для представлення предметної області. Розробник може адаптувати код до будь-якої складності. ► Розробники, які працюють над проектом, використовують спільну мову програмування, що полегшує внутрішню комунікацію в команді та підтримку коду.	► Код часто є занадто абстрактним і технічним для експертів предметної області. Вони не розуміють синтаксису, парадигм, бібліотек, що призводить до значного семантичного розриву між тим, як експерт бачить проблему, і тим, як вона реалізована в коді, через що експертам складно перевірити, чи точно програма відповідає їхнім бізнес-вимогам, дивлячись на код. Комунікація часто відбувається через проміжні артефакти (діаграми, специфікації) або інтерфейс користувача. ► Точність реалізації доменної логіки повністю залежить від того, наскільки добре розробник зрозумів вимоги експерта і як він їх імплементував за допомогою мови, що збільшує ризик невідповідності.
Технологічний аспект	
► Мова дозволяє будувати складні та масштабовані архітектури програмного забезпечення, використовуючи різні архітектурні патерни (наприклад, мікросервіси, моноліти, клієнт-сервер) та інтегруючи різноманітні технології. ► Програмісти мають високий ступінь контролю над тим, як саме програма виконується, що дозволяє тонко налаштувати її за для забезпечення продуктивності та керування ресурсами. ► Мова, як правило, оснащена розвиненими інструментами розробки (IDE, налагоджувачі, профайлери, системи контролю версій), які значно спрощують процес розробки, налагодження та підтримки коду.	► Для автоматизації специфічних доменних завдань необхідним є написання значної кількості коду вручну. Це стосується як бізнес-логіки, так і інфраструктурних аспектів. ► Гнучкість може призвести до написання надмірно складного або такого, що важко читається, коду. Без належного архітектурного підходу та стандартів кодування, великі проекти можуть стати кошмаром для підтримки та подальшого розвитку. ► Хоча існують численні бібліотеки, відсутність абстракцій специфічних для домену може спонукати розробників створювати власні “велосипеди” для вирішення типових доменних проблем, що може бути не ефективним або не надійним.
Аспект “порогу входження” та ефективності використання	
► Завдяки величезній популярності, мови загального призначення мають потужні методичну підтримку: тисячі книг, онлайн-курсів, посібників, документації, активних форумів та спільнот. Це робить їх доступними для вивчення та освоєння для початківців. ► Існує величезна кількість готових компонентів (бібліотек, фреймворків) для GPL, які дозволяють розробникам не винаходити колесо заново, прискорюючи розробку та дозволяючи зосередитися на унікальній логіці проекту. ► Володіння популярними мовами загального призначення робить суттєво підвищує шанси розробника на ринку праці, оскільки ці мови використовуються в більшості індустрій.	► Хоча почати вивчати мову легко, повне освоєння всіх її можливостей, парадигм, нюансів продуктивності та безпеки може зайняти дуже багато часу та зусиль. Обсяг знань, що накопичується, величезний. ► Велика кількість доступних бібліотек, фреймворків та інструментів може призвести початківців до “паралічу вибору” – незрозуміло з чого почати або яке рішення є найкращим в конкретній ситуації. ► Екосистеми популярних мов загального призначення постійно еволюціонують, з’являються нові версії мови, фреймворки, кращі практики. Це вимагає від розробників постійного навчання та адаптації.

програми. Спроби реалізувати обробку персистентних даних безпосередньо за допомогою GPL стикалися з серйозними проблемами:

1. ручне управління файлами – програмістам доводилося вручну керувати файлами (відкривати, закривати, читати/записувати, обробляти помилки введення/виведення), що є низькорівневою та складною задачею, з високим ризиком помилок програмування;
2. складність обробки зв’язків між даними – в мовах загального призначення відсутні вбудовані механізми для ефективного представлення та маніпулювання складними

зв'язками між даними, що є ключовим для, наприклад, реляційних баз даних. Розробникам доводилося кожний раз імплементувати власну логіку зв'язків та індексації;

3. забезпечення конкурентного доступу – гарантування цілісності даних при одночасному доступі до них з різних частин програми або від різних користувачів (блокування, транзакції) є надзвичайно складною задачею для ручної реалізації засобами мов загального призначення;
4. відсутність стандартизованих запитів – кожен раз потрібно було писати власний код мовою загального призначення для пошуку, фільтрації та агрегації даних, що було трудомістко, повільно і призводило до помилок.

Подолання цих труднощів було досягнуто концепцією *бази даних* – окремого від програми сховища персистентних даних з власним API¹. Бази даних взяли на себе відповідальність за зберігання, індексування, безпеку, конкурентний доступ та цілісність даних.

Але справжнім проривом стала поява спеціалізованих мов для взаємодії з базами даних. Наприклад, для реляційних баз даних була розроблена мова SQL (Structured Query Language). SQL є яскравим прикладом предметно-орієнтованої мови: вона не є повною за Тьюрінгом, але надає високоабстраговані та ефективні засоби для виконання операцій (вибірка, вставка, оновлення, видалення) над даними у реляційній моделі. Використання SQL дозволило:

- значно підвищити продуктивність розробників – замість тисяч рядків коду для обробки даних мовою загального призначення операції специфікуються кількома рядками мови SQL;
- покращити читабельність та прозорість коду – код мовою SQL є більш зрозумілим для експертів з обробки даних, що робить цей код легшим для підтримки;
- делегувати складну логіку системі управління базою даних (СУБД) – управління транзакціями, конкурентним доступом, відновленням даних після збоїв повністю перекладається на СУБД та її внутрішні механізми, керовані SQL.

Цікавим для мови SQL є перевірка семантичної коректності запитів (*queries*) – основних лінгвістичних конструкцій цієї мови. Семантично коректні запити в результаті свого виконання утворюють скінченні таблиці. Такі запити називають безпечними. Однак, синтаксис мови дозволяє будувати запити, які не є безпечними – їх виконання не завершується. На жаль, проблема розпізнавання небезпечних запитів є лише напіврозв'язною, а не розв'язною. Проте, Дж. Ульман знайшов властивість запитів (так звану *стратифікованість*), яка є достатньою для безпечності запиту та розв'язною. Практична значущість властивості запиту бути стратифікованим полягає в тому, що таким є майже всі “розумні”. Саме тому в більшості реляційних СУБД реалізований і ввімкнутий за умовчанням механізм перевірки, чи є запит стратифікованим. Проте цей механізм може бути вимкнений адміністратором СУБД.

Таким чином, приклад з обробкою персистентних даних яскраво демонструє, як слабкі сторони мов загального призначення (складність низькорівневої роботи, відсутність доменних абстракцій, труднощі з автоматичною перевіркою коректності) долаються за рахунок створення спеціалізованих систем та, що найважливіше, предметно-орієнтованих мов, які дозволяють ефективно працювати в дуже конкретній, але критично важливій предметній області (*домени*).

Саме це є головним мотивом розробки предметно-орієнтованих мов (англійською *Domain-Specific Languages, DSL*) – мов, пов'язаних з конкретним доменом.

¹Application Programming Interface – сукупність програмних засобів, що дозволяють взаємодіяти з окремим програмним комплексом.

5 Предметно-орієнтовані мови

Характеризуючи мови загального призначення, ми зазначали, що їхньою ключовою властивістю є повнота за Тьюрингом, тобто за допомогою такої мови можна реалізувати будь-який алгоритм. Однак реальна практика розробки програмних систем показує, що такий рівень універсальності мови є надлишковим для багатьох реальних проєктів. Мови загального призначення, незважаючи на свою універсальність та гнучкість, мають певні слабкі сторони (див. Табл.1), які впливають на якість та ефективність розробки програмних систем.

Приклад з обробкою персистентних даних (див. попередній параграф), яскраво демонструє ці слабкості мов загального призначення. Спроби реалізувати обробку персистентних даних безпосередньо за допомогою мов цього класу, стикалися з серйозними проблемами. Вирішення цих проблем було забезпечено завдяки концепції баз даних та, що найважливіше, появі спеціалізованих мов для взаємодії з ними. Саме це є головним мотивом розробки предметно-орієнтованих мов (*Domain-Specific Languages, DSL*) – мов, пов'язаних з конкретною предметною областю (доменом).

5.1 Які мови називають предметно-орієнтованими?

Предметно-орієнтована мова (DSL) – це комп'ютерна мова, спеціалізована для конкретної прикладної області. На відміну від мов загального призначення, які є універсальними і не залежать від предметної області, предметно-орієнтована мова створюється для вирішення проблем у конкретній предметній області і, як правило, не призначена для вирішення проблем поза нею, хоча технічно це може бути можливим. Предметно-орієнтована мова використовує термінологію та концепції, які вже знайомі фахівцям відповідної предметної області, що робить код легшим для читання та написання.

Простіші предметно-орієнтовані мови, особливо ті, що використовуються однією програмою, іноді неформально називають "малими мовами".

Розмежування між мовами загального призначення та предметно-орієнтованими мовами не завжди чітке, оскільки мова може мати спеціалізовані функції для певного домену, але бути ширше застосовною, або навпаки, бути в принципі здатною до широкого застосування, але на практиці використовуватися переважно для конкретного домену.

Прикладом може слугувати мова COBOL (**CO**mmon **B**usiness **O**riented **L**anguage), яка є мовою загального призначення, хоча її синтаксис орієнтований на розуміння фахівцями в економічній галузі, а не у програмуванні.

Предметно-орієнтовані мови можна поділити на *мови розмітки*, *мови моделювання* (інша назва – *мови специфікації*) та *мови програмування*.

5.2 Чому використовують предметно-орієнтовані мови?

Використання предметно-орієнтованої мови є ключовою частиною *доменної інженерії*, що дозволяє використовувати мову, яка найкраще відповідає домену проєкта. Створення предметно-орієнтованої мови може бути виправданим, якщо вона дозволяє виразити певний тип проблеми або рішення більш чітко і виразно, ніж мови загального призначення, при тому, що цей тип проблеми з'являється достатньо часто.

Відзначимо основні переваги предметно-орієнтованих мов.

Підвищена виразність у специфічному домені. Предметно-орієнтовані мови дозволяють виражати рішення в ідіомах та на рівні абстракції проблемної області. Вони надають механізми абстрагування, що дозволяють розробникам працювати з високорівневими концепціями, не заглиблюючись у дрібні деталі програмної чи апаратної архітектури. Це

робить їх інтуїтивно зрозумілішими та легшими у використанні для фахівців у цій галузі. Наприклад, згадуваний раніше SQL дозволяє взаємодіяти з базами даних за допомогою простих запитів замість складної логіки програмування. Код, написаний за допомогою предметно-орієнтованої мови, стає зрозумілим для експертів предметної області, що полегшує його підтримку та перевірку на відповідність бізнес-вимогам (подолання семантичного розриву).

Збільшення продуктивності та ефективності. Предметно-орієнтовані мови значно підвищують продуктивність розробників, оскільки дозволяють специфікувати операції кількома рядками коду порівняно з тисячами рядків коду мовою загального призначення. Вони автоматизують повторювані процеси кодування, зменшуючи кількість помилок та заощаджуючи час. Наприклад, при обробці персистентних даних, SQL значно підвищив продуктивність розробників, делегувавши складну логіку системі управління базами даних (СУБД).

Спрощене обслуговування та читабельність. Завдяки вузькій спрямованості, предметно-орієнтовані мови усувають зайву складність, роблячи код чистішим, більш читабельним та легшим для підтримки. Вони допомагають зменшити кількість помилок у коді та підвищити ефективність команди розробників.

Зниження "порогу входження" для експертів. Добре спроектовані предметно-орієнтовані мови не вимагають глибоких знань теорії та практики програмування. Ідея полягає в тому, що експерти домену самі можуть розуміти, валідувати, модифікувати і навіть розробляти програми такою мовою. Це дозволяє звільнити спеціаліста у певній доменній області від необхідності вивчати програмування, а програміста – від необхідності розбиратися в предметі.

5.3 Особливості предметно-орієнтованих мов та принципи їх дизайну

Головними особливостями предметно-орієнтованої мови є те, що вона, по-перше, оперує виключно з поняттями, які відповідають певному домену, предметній області, а по-друге, забезпечує комунікацію між експертами цього домену та розробниками програмного забезпечення для нього.

Таким чином, добре спроектовані предметно-орієнтовані мови мають спеціалізований синтаксис, адаптований до конкретного домену. Вочевидь, вони є не універсальними, вузько спрямованими, що робить їх надзвичайно ефективними для завдань, пов'язаних з доменом, але погано придатними, а іноді й зовсім непридатними, для завдань поза цим доменом.

Предметно-орієнтовані мови також значно виразніші у своєму домені у порівнянні з мовами загального призначення, забезпечуючи мінімальну надлишковість для завдань домену.

Багато предметно-орієнтованих мов не є повними за Тьюрингом, тобто за їхньою допомогою не можна написати будь-яку програму. Це обмеження часто вноситься розробниками навмисно задля спрощення мови та глибшого, включно з семантичним, аналізу та оптимізації. Наприклад, SQL не є повною за Тьюрингом мовою, проте надзвичайно ефективним інструментом для операцій з базами даних.

Важливою особливістю предметно-орієнтованих мов є їх орієнтація на "що? а не на "як?". Вони часто сприяють декларативному програмуванню, описуючи бажаний результат, а не детальні кроки для його досягнення.

На останок зазначимо, що при розробці предметно-орієнтованої мови в першу чергу слід орієнтуватися на потреби не програмістів, а доменних експертів – синтаксис повинен бути розроблений так, щоб він був зручним для доменних експертів, використовувати їх

термінологію, навіть якщо це порушує програмістські конвенції. Пріоритетом має бути читабельність коду, а не програмістські традиції.

5.4 Два типи предметно-орієнтованих мов

Розрізняють два типи предметно-орієнтованих мов – *внутрішні* та *зовнішні*.

Внутрішні предметно-орієнтовані мови (*Embedded DSL, eDSL*). Такі мови реалізуються як бібліотеки або фреймворки всередині “хост” мови, яка є мовою програмування загального призначення.

До переваг таких мов можна віднести те, що вони використовують синтаксис, семантику та середовище виконання хост-мови, а також існуючі для неї інструменти (налагоджувачі, IDE, бібліотеки); крім того, вони швидше будуються.

Щодо реалізації мовних конструкцій, то вони часто створюються за допомогою патерну “будівельник”(builder) та ланцюжкового виклику методів (method chaining), де імена методів відповідають словнику домену.

Приклади внутрішніх предметно-орієнтованих мов:

- SQLAlchemy “Core”(SQL eDSL у Python) [1];
- jOOQ (SQL eDSL у Java) [2];
- “синтаксис методів”LINQ (SQL eDSL у C#) [3];
- тестові фреймворки з fluent-інтерфейсами (наприклад, FluentAssertions для .NET [4], Chai для Java Script / Node.js [5] або AssertJ для Java [6]) також є успішними прикладами внутрішніх предметно-орієнтованих мов.

Зовнішні DSL. Мають повністю новий, незалежний синтаксис і зазвичай вимагають окремого інтерпретатора або компілятора.

Перевагами таких мов є те, що вони надають повний контроль над кожним аспектом мови, є більш гнучкими.

Однак, платою за ці переваги є те, що вони потребують побудови парсерів, механізмів обробки помилок та інших інструментів з нуля.

Приклади зовнішніх предметно-орієнтованих мов:

- TeX – добре відома мова однойменної редакторської системи [7];
- SQL – мова структурованих запитів до реляційних баз даних [8];
- CSS – мова каскадних таблиць стилів [9];
- Regex – мова регулярних виразів [10];
- Gherkin – мова використовується для документування вимог та визначення тестових сценаріїв [11].

6 Висновки

Підсумовуючи викладене, можемо стверджувати, що мова відіграє надзвичайно важливу роль як у людському пізнанні та спілкуванні, так і в контексті програмування. Для людини мова є невід’ємною частиною когнітивних процесів, дозволяючи *структурувати думки*,

абстрагуватися, зберігати і відтворювати інформацію, а також розвивати свідомість. Вона є унікальним інструментом комунікації, гнучким засобом адаптації, головним носієм культури, що об'єднує людей та сприяє передачі знань і координації дій. Саме це робить нас здатними до створення складних соціальних структур, науки, мистецтва та технологій.

У царині програмування мова відіграє роль *формального засобу взаємодії людини з комп'ютером*, будучи мостом між людським інтелектом та обчислювальною потужністю машин.

Ключові функції мов програмування включають:

1. управління комп'ютером;
2. забезпечення механізмів *абстрагування* для вирішення складних задач;
3. структурування даних;
4. забезпечення формальної специфікації та реалізації *алгоритмів*;
5. спрощення спілкування між програмістами;
6. управління ресурсами комп'ютера.

Весь сучасний технологічний світ, що базується на цифрових технологіях, не може функціонувати без систематизованого та логічного засобу створення цифрових рішень, яким є формальна мова. Загальна кількість мов програмування оцінюється в діапазоні від 8 000 до 9 000, і всі вони є формальними.

Існують два великих класи мов програмування – мови загального призначення та предметно-орієнтовані мови.

Мови загального призначення (General-Purpose Languages, GPL) характеризуються повнотою за Тьюрінгом, що дозволяє специфікувати будь-який алгоритм. Вони є універсальними та гнучкими, призначеними для вирішення широкого спектра задач у різних предметних областях. Приклади таких мов включають Python, Java, C++, JavaScript, C#, Go та Rust. Хоча GPL мають сильні сторони, такі як необмежена виразність, розвинені інструменти розробки та потужна методична підтримка, вони також мають слабкі сторони. Серед слабкостей GPL слід зазначити неможливість універсального автоматичного аналізу семантичних властивостей програм (наслідки теореми Райса), відсутність вбудованих механізмів для представлення доменної логіки, значний семантичний розрив між кодом та розумінням експертів предметної області, необхідність значного ручного кодування для доменних завдань та високий поріг освоєння всіх можливостей.

Ці слабкості GPL яскраво демонструє приклад обробки персистентних даних, де ручне управління файлами, складність обробки зв'язків, забезпечення конкурентного доступу та відсутність стандартизованих запитів створювали серйозні проблеми. Вирішенням стало впровадження баз даних та, головне, поява предметно-орієнтованих мов (Domain-Specific Languages, DSL), таких як SQL.

Предметно-орієнтована мова (DSL) є комп'ютерною мовою, спеціалізованою для конкретної прикладної області, що використовує термінологію та концепції, знайомі фахівцям цієї області. Використання DSL виправдане, коли потрібно виразити певний тип проблеми або рішення чіткіше та виразніше, ніж мови загального призначення, і коли цей тип проблеми виникає часто.

Основними перевагами DSL є підвищену виразність у специфічному домені, що дозволяє працювати з високорівневими концепціями та робить код зрозумілим для експертів; збільшення продуктивності та ефективності розробників завдяки автоматизації повторюваних процесів та природному для предметної області представленню складної логіки; спрощення

обслуговування та читабельності коду через його вузьку спрямованість; зниження "порогу входження" для експертів, що дозволяє їм розуміти, валідувати та навіть розробляти програми без глибоких знань програмування.

Особливістю DSL є те, що вона оперує виключно поняттями певного домену та забезпечує комунікацію між експертами та розробниками. Вона має *спеціалізований синтаксис*, який є вузько спрямованими, надзвичайно ефективними для доменних завдань. Часто DSL не є повною за Тьюрінгом, що є свідомим обмеженням для спрощення мови та глибшого аналізу. DSL часто відповідає радше *декларативному стилю програмування*, орієнтуючись на формулюванні того, що потрібно зробити, а не на тому, як це зробити. Дизайн DSL в першу чергу враховує потреби доменних експертів, а не програмістів.

Розрізняють два типи предметно-орієнтованих мов: внутрішні DSL (Embedded DSL, eDSL), які реалізуються як бібліотеки або фреймворки всередині "хост" мови загального призначення, використовуючи її синтаксис та інструменти, та зовнішні DSL, які мають повністю новий, незалежний синтаксис і вимагають окремого інтерпретатора або компілятора, надаючи повний контроль над мовою за рахунок побудови інструментів з нуля.

Таким чином, можна резюмувати, що вибір в рамках конкретного проєкту на користь використання мови загального призначення, або на користь створення предметно-орієнтованої мови, є надзвичайно складним рішенням, яке має ключове значення для успішності проєкту. В той же час, сучасний фахівець у галузі комп'ютерних наук має бути готовий до створення мови, орієнтованої на певну предметну область, та всієї технологічної інфраструктури для використання створеної мови.

Література

1. *SQLAlchemy Project*. SQLAlchemy – The Database Toolkit for Python. — 2025. — Дата доступу: 07.08.2025. <https://www.sqlalchemy.org/>.
2. *Data Geekery GmbH*. jOOQ – The easiest way to write SQL in Java. — 2025. — Дата доступу: 07.08.2025. <https://www.jooq.org/>.
3. *Microsoft*. Get started with LINQ in C#. — 2025. — Дата доступу: 07.08.2025. <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>.
4. *Dennis Doomen and Contributors*. FluentAssertions – A set of .NET extension methods that allow you to more naturally specify the expected outcome of a TDD or BDD-style test. — 2025. — Дата доступу: 07.08.2025. <https://fluentassertions.com/>.
5. *Chai.js*. Chai – BDD / TDD assertion library for node and the browser. — 2025. — Дата доступу: 07.08.2025. <https://www.chaijs.com/>.
6. *J. Costigliola and AssertJ Contributors*. AssertJ – Fluent assertions for Java. — 2025. — Дата доступу: 07.08.2025. <https://joel-costigliola.github.io/assertj/>.
7. *TeX User Group*. TeX Users Group (TUG). — 2025. — Дата доступу: 07.08.2025. <https://www.tug.org/>.
8. *International Organization for Standardization*. ISO 8601-1:2019, Date and time representations — Part 1: Basic rules. — ISO, 2019. — Дата доступу: 07.08.2025. <https://www.iso.org/standard/76583.html>.
9. *W3SchoolsUA*. Уроки та довідник з CSS. — 2025. — Дата доступу: 07.08.2025. <https://w3schoolsua.github.io/css/index.html>.
10. *Data Life UA*. Регулярні вирази (RegExp): навіщо, як і для чого? — 2024. — Дата доступу: 07.08.2025. https://data-life-ua.com/coding/reg_exp-rehuliarni-vyrazy-navishcho-yak-i-dlia-choho/.
11. *Gherkin UFT*. Gherkin. — 2025. — Дата доступу: 07.08.2025. <https://www.gherkinuft.com/gherkin/>.